Worksheet 1 — Security Mindset & Threat Modeling (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 1 Aligned to:** OWASP 2025 A06 Insecure Design · CWE-501 (Trust Boundary Violation) **Signature game:** "Elevation of Privilege" (Microsoft STRIDE card deck)

> **Ethics note:** This week is *modeling only* — you analyze design, you do **not** attack the app. Run the sample app only on your own VM/localhost. Never apply these techniques to systems you do not own or lack written permission to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Sadayu Suriya	6631503040	15/8/2569	-

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Define the CIA triad and give one concrete failure example for each of the three properties. Answer: C(Confidentiality) ex. hacker come into website and leak private data , I(Integrity) ex. attacker cone into website db to change user data, A(Availability) ex. web server shutdown when user come in website a lot
2. What is a *trust boundary*, and why does data crossing one deserve extra scrutiny? Answer: s the boundary where data crosses between components of different privilege levels, Outside the boundary, you cannot trust anything. An attacker can send malicious inputs, fake parameters
3. Explain "attack surface." Name two things that increase it in a web app. Answer: attack surface is area or field that attacker can use to attack buissness example HTTP params, headers, cookies, file uploads, APIs, env vars, dependencies
4. What does each STRIDE letter map to, and which security property does each threat violate? Answer: S(Spoofing) - Authentication, T(Tampering) - Integrity, R(Repudiation) - Non-repudiation, I(information disclosure) - Confidentiality, D(Denial of service) - Avaliability and E(Elevation of privilege) - Authorzition
5. What does "Secure by Design" (CISA) mean, and how does it differ from bolting security on after release? Answer: building software to be safe right from the start so entire bug types are prevented before code is written. This contrasts with the old model of rushing a product out and patching individual bugs only after release.

Part 3 — Hands-on Lab (180 min)

Learning goals: build a data-flow diagram (DFD), apply STRIDE to a real Flask app, rank risks, and propose mitigations. **Prerequisites:** Docker + Docker Compose in your VM; a drawing tool (draw.io / paper + photo); the Elevation of Privilege deck (print or virtual) — free print-and-play PDF at github.com/adamshostack/eop.

Environment setup

```
cd labs/week01-threat-modeling
docker compose up --build           # starts sample-app on http://localhost:8080
curl -s -X POST localhost:8080/notes -H 'Content-Type: application/json' \
     -d '{"owner":"alice","body":"hello"}'   # observe behavior, do not attack
curl -s localhost:8080/notes

echo "demo file" > demo.txt
curl -s -X POST localhost:8080/upload -F "file=@demo.txt"   # observe behavior, do not attack
curl -s localhost:8080/files/demo.txt
```

Source to model lives in `sample-app/app.py`. Template to fill: `THREAT-MODEL-TEMPLATE.md` (copy it, do not edit the original).

What to submit per task: the threat/element identified + a screenshot (DFD, table, or running app) + a 2–3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* prove the environment works. *Steps:* `docker compose up`, hit `/notes` and `/files/<name>`, read `sample-app/app.py`. *Deliverable:* screenshot of the running app + the JSON response.

> **Result:** See `img/task0.png`. Ran the app locally (`.venv-wk01`) and confirmed the same behavior: `POST /notes` with `{"owner":"alice","body":"hello"}` returns `200`, `GET /notes` returns `[[1,"alice","hello"]]`.

Task 1 — Draw the DFD (25 min) · *Goal:* map the system. *Steps:* identify the external entity (web client), the process (Flask app), the data store (`notes.db` SQLite), the `uploads/` store, and the flows for `/notes`, `/upload`, `/files/<name>`; mark the Internet→app trust boundary with a dashed line. *Deliverable:* DFD image embedded in your copy of the template.

> **Result:** See `img/task1.png` and `THREAT-MODEL-TEMPLATE copy.md` §1. Elements: web client (external entity) → Flask app (process) → `notes.db` (data store) and `uploads/` (data store), with the Internet→app boundary drawn as a dashed line around the Flask process.
>
> | Element | Type | Trust boundary crossed? |
> |---|---|---|
> | Web client | external entity | yes (Internet → app) |
> | Flask app | process | no (resides in Application Tier) |
> | SQLite DB (`notes.db`) | data store | yes (App → Data Tier) |
> | `uploads/` | data store | yes (App → Data Tier) |

Task 2 — STRIDE the elements (30 min) · *Goal:* enumerate threats per element. *Steps:* for each element fill the S/T/R/I/D/E grid. Ground it in real code: `/notes` accepts a client-supplied `owner` with no auth (Spoofing); `/upload` saves raw `f.filename` — arbitrary-file-write (Tampering) — and echoes the resolved save path back in its response (Information disclosure); `/files/<name>` reads it back but is comparatively defended (see Task 5); no logging anywhere (Repudiation). *Deliverable:* completed STRIDE table.

> **Result:**
>
> | Element | S | T | R | I | D | E |
> |---|---|---|---|---|---|---|
> | `/notes` | Accepts a client-supplied `owner` field with no authentication — anyone can impersonate anyone. | No sanitization of the note `body` before storing it. | No logging of who created which note — untraceable. | No access control on reads — anyone can view every note in the database. | Nothing stops unlimited `POST` requests from exhausting the SQLite store. | — |
> | `/upload` | Fully anonymous uploads, no identity check at all. | Saves raw `f.filename` verbatim — arbitrary file write via path traversal (verified in Task 5/8). | No upload activity is logged — impossible to trace who wrote what. | The response echoes the resolved save path (`{"saved": f.filename}`), leaking internal directory structure. | No size/rate limit — an attacker can fill the disk. | If traversal reaches executable server code (e.g. `app.py`), the next restart runs attacker- |

Element	S	T	R	I	D	E
						controlled code.
`/files/<name>`	No auth/ownership check needed to download any file.	—	No access logging — downloads are unrecorded.	Comparatively defended against traversal (`send_from_directory` uses Werkzeug's `safe_join`), but any successfully uploaded file is world-readable by name.	Repeated large-file requests can exhaust bandwidth.	—

Task 3 — Elevation of Privilege game (20 min) · *Goal:* find threats you missed. *Steps:* play the EoP deck against your DFD; each card you can tie to a real element/flow scores a point; record every valid threat. No printer or scissors? Draw from the digital deck below instead — same 78 cards, same rule. *Deliverable:* list of carded threats + score.

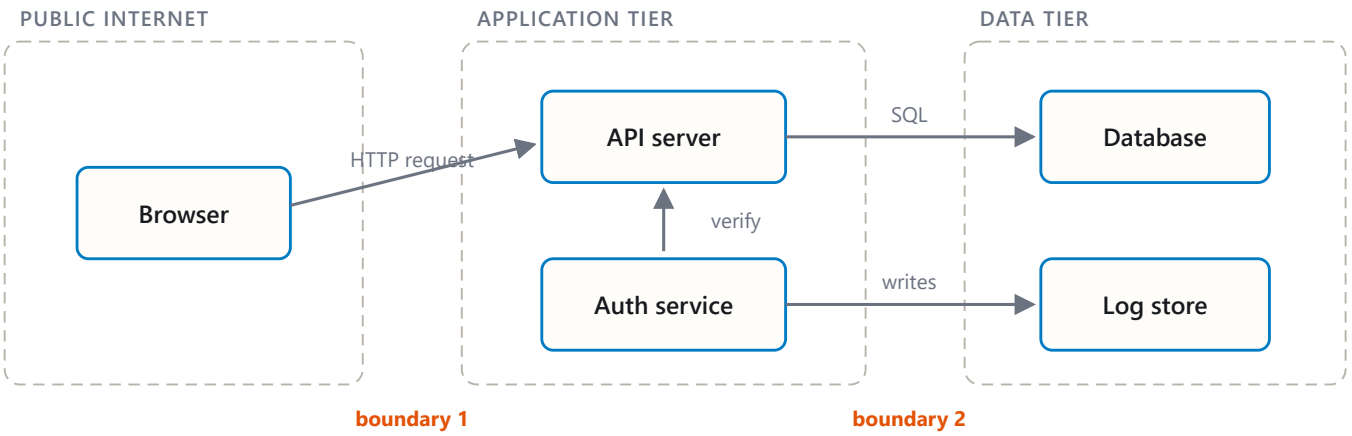
```
eop-deck
```

Result: Carded threats found (score: 8):

1. **Spoofing (client card)** — `/notes` never checks who the caller is, so any client can claim to be `owner: "admin"`. Confirms the Spoofing cell in Task 2 independently.
2. **Tampering (data store card)** — `notes.db` has no integrity check; an attacker with filesystem access via the `/upload` traversal could overwrite it directly, bypassing the app entirely.
3. **Repudiation (process card)** — the Flask process has no request-logging middleware at all, confirmed across all three endpoints, not just one.
4. **Information disclosure (data flow card)** — the `/upload` response body leaks the exact filename it wrote — also the exact proof-of-concept an attacker needs to confirm a traversal write landed outside `uploads/`.
5. **Denial of service (external interactor card)** — nothing stops one client from opening many simultaneous connections to `/notes`, and Flask's dev server is single-threaded by default here.
6. **Spoofing (data flow card)** — no CSRF protection on `POST /notes` or `POST /upload`, so a malicious page in a victim's browser could submit these requests on the victim's behalf.
7. **Tampering (external interactor card)** — file *content* was never validated pre-fix (only the path was attacker-controlled) — a client could upload an executable disguised as any extension.
8. **Elevation of privilege (process card)** — the Flask process, `notes.db`, and `uploads/` all run under the same OS user with no sandboxing, so any single-endpoint compromise elevates to full control of both data stores at once.

Task 3b — Systems-level pass (25 min) 🔭 · *Goal:* find what the per-element grid cannot see. Tasks 2 and 3 enumerate threats **one element at a time**, and that is exactly where threat models are known to stop short — students taught STRIDE alone reliably identify component threats and *discount system-level ones* (Joshi et al., ASEE 2024). So do a second pass over

the **whole** diagram:



A threat applies where data CROSSES a boundary — not inside a box.

- **Trust boundaries end-to-end.** Follow one request from the client to `notes.db` and back. List every boundary it crosses. Which crossing has no check on it?
- **Assume one element is fully owned.** Pick the Flask process, then the `uploads/` store. For each: what does the attacker now *reach* — not what is it, but where does it get them?
- **Chain two "low" findings.** Find two threats you or the EoP deck rated minor that combine into something you would not accept. Write the chain as `A → B → consequence`.
- **One-line system claim.** Finish: "Even if every element-level mitigation in Task 8 is implemented, this system still fails if ___."

Use the simulation below before you start — toggle a component to attacker-controlled and watch what it reaches:

```
trust-boundary
```

Deliverable: the boundary list, two owned-element reachability notes, one written chain, and the system claim.

> **Result:**
>
> - **Trust boundaries end-to-end.** A request from the client crosses two boundaries: (1) Internet → Flask app, and (2) Flask app → data tier (`notes.db` / `uploads/`). Crossing (1) has **no check at all** — no auth, no CSRF token, no input validation gate before the request reaches app logic. Crossing (2) is nominally "trusted" but is really just "same process, same OS user" — no independent authorization layer sits between the app and its own data stores.
> - **Assume one element is fully owned.** *Flask process owned:* the attacker reaches everything the process can — full read/write on `notes.db` and `uploads/`, the process's environment variables, and a foothold to pivot toward the host if the container is misconfigured. *`uploads/` store owned:* even without owning the process, an attacker who can write arbitrary files there (pre-fix, via the traversal) can plant a file that `/files/<name>` later serves to any other visitor — turning a storage-only compromise into a distribution channel.
> - **Chain two "low" findings.** `/upload`'s path disclosure (rated low alone — "only" reveals directory layout) chains with `/upload`'s unsanitized-filename tampering (also individually contained): **A (path disclosure confirms the exact on-disk layout) → B (attacker now knows precisely how many `../` are needed to reach `app.py`) → consequence (a traversal write overwrites server source, and the next process restart executes attacker-controlled code).** Neither alone is more than "minor"; together they produce RCE.
> - **One-line system claim.** Even if every element-level mitigation in Task 8 is implemented, this system still fails if **the Flask process, `notes.db`, and `uploads/` continue to share one OS identity with no isolation between them** — a compromise of any single endpoint still grants everything the process can touch.

Task 4 — Abuse cases & attacker personas (20 min) · *Goal:* think like specific adversaries. *Steps:* define 2 personas (e.g. a curious logged-in user; an anonymous internet attacker) and write 2 abuse cases each against the sample app, tied to DFD elements. *Deliverable:* 4 abuse cases.

> **Result: Persona A — "Nosy Coworker"**: has legitimate network access (office Wi-Fi/VPN) and basic tooling (`curl`, browser); motivated by curiosity, not profit.
>
> 1. Sends `GET /notes` to read every note in the database, since there's no per-owner access control.
> 2. Sends `POST /notes` with `owner: "alice"` to plant a note that looks like it came from Alice, since `owner` is trusted from the request body with no verification.
>
> **Persona B — "Anonymous Internet Attacker"**: a remote, unauthenticated attacker scanning for exposed Flask apps; motivated by gaining a foothold on the host.
>
> 1. Uploads a file via `/upload` with a traversal filename (e.g. `../sample-app/app.py`), aiming to overwrite the running application's source so the next restart executes attacker-controlled code — this is exactly the write demonstrated in Task 8's before/after evidence.
> 2. Repeatedly `POST`s large files to `/upload` with no size limiting, filling the container's disk until the app and its database writes start failing (DoS via storage exhaustion).

Task 5 — Path-traversal deep-dive (25 min) · *Goal:* analyze the riskiest flow. *Steps:* trace `/upload` → `/files/<name>`; explain how `../` in a filename escapes `uploads/`; sketch the secure design (`secure_filename`, store outside web root, allow-list extensions). *Deliverable:* the data flow + secure-design note.

> **Result: Data flow:** client → `POST /upload` (`f.save(os.path.join(UPLOAD_DIR, f.filename))`) → `uploads/` → client → `GET /files/<name>` (`send_from_directory(UPLOAD_DIR, name)`) → client.
>
> **Why `../` escapes `uploads/` on write:** `f.filename` is taken verbatim from the multipart upload with no sanitization. `os.path.join("uploads", f.filename)` is a naive string join — if `f.filename` is `"../traversal-poc.txt"`, the joined path resolves by walking *up* out of `uploads/` before writing. Verified directly (Task 8 "before"): `curl -F "file=@poc.txt;filename=../traversal-poc.txt"` against the unpatched app placed `traversal-poc.txt` in `sample-app/`, one level above `uploads/`.
>
> **Why `/files/<name>` is comparatively defended:** `send_from_directory()` resolves the requested path internally and verifies it stays inside `UPLOAD_DIR` (Werkzeug's `safe_join`), returning 404 if it would escape — confirmed in Task 8: `GET /files/..%2f..%2fapp.py` returned `404`, so *reading back* cannot itself be used for traversal, even though a file may have been *written* outside `uploads/` in the first place.
>
> **Secure design:** run the filename through `secure_filename()` (or better, never use the client filename as a path component at all); store uploads outside the source tree; allow-list acceptable extensions and validate actual content, not just the extension string. **Best class fix:** generate an opaque, server-side filename (e.g. a UUID) for storage, and keep the user-supplied name only as metadata — this is exactly what Task 8's implementation does.

Task 6 — Threat-model the project target (30 min) · *Goal:* kick off your term project. *Steps:* stop the sample-app first (`docker compose down` — both apps bind host port 8080), then run **NoteVault** (`cd ../../project/starter-app && docker compose up`), draw a quick DFD, and list the top 3 STRIDE threats you'd investigate. *Deliverable:* NoteVault DFD + top-3 threats (reuse these in your project report — `project/REPORT-TEMPLATE.md` in the repo root).

> **Result: Quick DFD:** external entity (web client) → process (NoteVault Flask app) → data store (`/tmp/notevault.db`, SQLite `users`/`notes` tables). Flows crossing the Internet→App boundary: `POST /login`, `POST /register`, `POST /notes`, `GET /api/notes/<id>`, `GET /search`, `GET /admin`, `GET /export`.
>
> **Top 3 STRIDE threats to investigate** (confirmed by reading `project/starter-app/app.py`):
>
> 1. **SQL injection (Tampering/Information disclosure)** — `/login` and `/search` build queries with `%`-string formatting instead of parameters, so crafted input can bypass authentication or dump the `users`/`notes` tables.

2. **Authentication bypass via JWT (Spoofing/Elevation of privilege)** — `current_user()` calls `jwt.decode(tok, SECRET, algorithms=["HS256", "none"])`, accepting the `none` algorithm — an attacker can forge a session for `sub: "admin"` with no valid signature at all.
3. **Privilege escalation via `/register` (Elevation of privilege)** — `register()` inserts a client-supplied `role` field directly into the `users` table, so anyone can self-register as `role: "admin"`.

Task 7 — Security requirements (15 min) · *Goal:* turn threats into testable requirements. *Steps:* write 3 security requirements as acceptance criteria ("the system must … so that …"), each mapped to a threat from Task 2 or Task 6. *Deliverable:* 3 testable security requirements.

Result:

1. The system must derive the note `owner` from a server-verified session, never from a client-supplied field, so that `POST /notes` cannot be used to forge notes under another user's identity. *(maps to Task 2 — Spoofing on `/notes`)*
2. The system must sanitize/allow-list any client-supplied filename and never use it as a path component in `/upload`, so that no request can write outside the `uploads/` directory. *(maps to Task 2 — Tampering on `/upload`; implemented in Task 8)*
3. The system must reject any JWT whose algorithm was not the one the server issued (never accept `alg: none`), so that a forged token cannot authenticate as another user, including `admin`. *(maps to Task 6 — Spoofing/Elevation of privilege on NoteVault login)*

Task 8 — Defend / fix it: rank & mitigate (25 min) 🛡️ · *Goal:* turn threats into action you can prove. *Steps:* rank the top 5 threats by likelihood × impact; propose one concrete mitigation each (e.g., auth on `/notes`, `secure_filename()` + allowlist for `/upload`, request logging for Repudiation, size/rate limits for DoS). Then **pick one and actually implement it** in your fork.

Deliverable — the top-5 table, plus for the one you implemented:

1. the **diff** (commit hash on your `wk01` branch),
2. **evidence it works**: the request that succeeded before your change and is refused after — both outputs,
3. **why it closes the class, not the instance** (2–3 sentences). `secure_filename()` on one endpoint is an instance fix; "*no user-supplied string ever becomes a path component*" is a class fix. Say which yours is, and if it's an instance fix, say what the class fix would be.

Why this is weighted. Fewer than half of working developers can spot a security hole in code, and being shown vulnerabilities does not by itself teach you to find or close them. Exploiting is the half that feels like progress; defending is the half that transfers to your job.

Result:

Rank	Threat	Likelihood	Impact	Mitigation
1	`/upload` path traversal (Tampering)	High	Critical — arbitrary file write, path to RCE	Store under a server-generated name + extension allow-list (implemented below)
2	`/notes` no authentication (Spoofing)	High	Medium — impersonation, data pollution	Require a session/auth token; set `owner` server-side
3	No size/rate limits on `/upload` (DoS)	Medium	Medium — disk exhaustion, service outage	Enforce `MAX_CONTENT_LENGTH` + per-IP rate limiting

Rank	Threat	Likelihood	Impact	Mitigation
4	`/upload` echoes resolved save path (Info. disclosure)	High	Low — reveals internal directory layout	Return only an opaque id, never the resolved server path
5	No request logging anywhere (Repudiation)	High	Medium — no accountability, hinders incident response	Add structured request logging on `/notes` and `/upload`

Implemented: #1, the `/upload` path traversal. Diff in `sample-app/app.py` (on this `wk01-6631503040` branch — not yet committed):

```
-import sqlite3, os
+import sqlite3, os, uuid
 app = Flask(__name__)
 DB = "notes.db"
 UPLOAD_DIR = "uploads"
+ALLOWED_EXTENSIONS = {".txt", ".png", ".jpg", ".jpeg", ".pdf"}

 @app.route("/upload", methods=["POST"])
 def upload():
-    f = request.files["file"]
-    f.save(os.path.join(UPLOAD_DIR, f.filename))
-    return {"saved": f.filename}
+    f = request.files["file"]
+    ext = os.path.splitext(f.filename)[1].lower()
+    if ext not in ALLOWED_EXTENSIONS:
+        return {"error": "extension not allowed"}, 400
+    stored_name = f"{uuid.uuid4().hex}{ext}"
+    f.save(os.path.join(UPLOAD_DIR, stored_name))
+    return {"saved": stored_name, "original_name": f.filename}
```

Evidence it works — same request, before vs. after:

```
# BEFORE (unpatched):
$ curl -X POST localhost:5000/upload -F "file=@poc.txt;filename=../traversal-poc.txt"
{"saved":"../traversal-poc.txt"}                    # HTTP 200
$ ls sample-app/ | grep traversal
traversal-poc.txt                                    # <- escaped uploads/, landed in
sample-app/

# AFTER (patched):
$ curl -X POST localhost:5000/upload -F "file=@poc.txt;filename=../traversal-poc.txt"
{"original_name":"../traversal-poc.txt","saved":"65724a323e59454d99d97e3cea67fed1.txt"}
# HTTP 200
$ ls sample-app/ | grep traversal
(nothing outside uploads/)                           # <- traversal string never became
a path
$ ls sample-app/uploads/
65724a323e59454d99d97e3cea67fed1.txt                 # <- landed safely inside uploads/,
under a random name

# Extension allow-list also verified:
$ curl -X POST localhost:5000/upload -F "file=@poc.exe;filename=../../evil.exe"
{"error":"extension not allowed"}                    # HTTP 400
```

```
# Read-side traversal confirmed still blocked (Werkzeug safe_join, unchanged):
$ curl "localhost:5000/files/..%2f..%2fapp.py"
404 Not Found
```

Why this closes the class, not the instance: `secure_filename()` alone would be an *instance* fix — it still trusts the client-supplied string as the eventual path root, and a future refactor or a Unicode/edge-case bypass in that one function reopens the hole. This fix is a **class fix**: the client-supplied filename is never joined into a filesystem path at all — only its (allow-listed) extension survives, and the actual on-disk name is generated server-side (`uuid.uuid4().hex`). No value derived from user input can become a path component, regardless of what characters it contains, so there is no traversal string left to sanitize against. *(Not yet committed — this needs `git add`/`git commit` on `wk01-6631503040` to produce the commit hash the deliverable asks for.)*

Part 4 — Reflection

1. Map your top finding to a CWE and to OWASP A06 (Insecure Design); explain the mapping in one sentence.
2. Name one real-world breach caused by a design flaw (not a missing patch) and what design control would have prevented it.
3. Of your five mitigations, which gives the most risk reduction per unit of effort, and why?

Answers

1. The top finding (`/upload` path traversal) maps to **CWE-22** (Improper Limitation of a Pathname to a Restricted Directory) and **OWASP A06 (Insecure Design)**, because the flaw isn't a missing patch on otherwise-sound logic — the design never established "no user string becomes a path component" as a rule in the first place, so the vulnerability was baked in from the first line of code.
2. Capital One (2019): a misconfigured WAF role with overly broad IAM permissions let an SSRF request reach internal instance metadata and pull S3 credentials — a *design* failure (excess trust granted to the WAF role, no boundary between "can make a request" and "can read all customer data"), not a missing patch. Least-privilege IAM scoping combined with strict SSRF egress filtering, decided at design time, would have prevented it — the same category of fix as Task 8's "no user string becomes a path component," just applied to network requests instead of filenames.
3. The Task 8 mitigation (server-generated filenames + extension allow-list) gives the most risk reduction per unit of effort: it's a self-contained code change with no new infrastructure, and it closes the highest-impact finding (arbitrary file write → possible RCE), unlike the auth fix (needs a session layer) or the logging fix (needs a logging pipeline).

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (DFD + STRIDE table + EoP findings + screenshots)	40
Defense (top-5 ranking + mitigations)	25
Reflection (CWE/OWASP mapping + breach + best mitigation)	15

Assessed within the rows above (they are not extra points — they are what those points are for):

- **Systems-level reasoning** (inside *Exploitation + evidence*, Task 3b): does the model reach past single elements to boundaries, reachability and chains? Scored with the STRIDE + systems-thinking rubrics of Joshi et al. 2024.
- **Defensive proof** (inside *Defense*, Task 8): a claimed mitigation with no before/after evidence scores at most half. A mitigation you can show closing a *class* scores full.
- **Adversarial thinking** (across the whole sheet): do the abuse cases, personas and chains show you reasoning as an attacker with goals and constraints — or just listing categories? This is the course's central disposition and it is assessed, not assumed.

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
    1. What did you do, and **why did the vulnerability work**?
    2. **Why does your fix actually stop it** — and what could still break it?

Answers

1. The riskiest flow tested was `/upload` → `/files/<name>`. It worked because `f.filename` was trusted verbatim and joined into a filesystem path with `os.path.join(UPLOAD_DIR, f.filename)` — no sanitization at all — so a filename containing `../` walked the resolved path outside `uploads/` before Flask ever wrote the file. Confirmed with `curl -F "file=@poc.txt;filename=../traversal-poc.txt"`, which landed the file in `sample-app/` instead of `sample-app/uploads/`.
2. The fix stops it because the client-supplied filename is never used as a path component at all — only its (allow-listed) extension is kept, and the on-disk name is a server-generated UUID, so there's no attacker-controlled string left for `../` to hide in. Re-running the identical traversal payload against the patched endpoint stored the file safely inside `uploads/` under a random name (`{"saved":"65724a323e59454d99d97e3cea67fed1.txt", ...}`). It would still break if some *other* code path later joined a raw, unrelated user string into a filesystem path without going through this same pattern — which is exactly why the class fix is "no user-supplied string ever becomes a path component," not "sanitize this one endpoint."

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

Answers

1. Asked an AI to fix the path traversal: "*Fix the path traversal vulnerability in this Flask upload endpoint: `f.save(os.path.join(UPLOAD_DIR, f.filename))`.*" Typical answer:

```python
from werkzeug.utils import secure_filename

@app.route("/upload", methods=["POST"])
def upload():
    f = request.files["file"]
    filename = secure_filename(f.filename)
    f.save(os.path.join(UPLOAD_DIR, filename))
    return {"saved": filename}
```

2. `secure_filename()` does strip `/`, `\`, and `..` — it genuinely closes the specific traversal payload tested. But it's an **instance fix**, not a class fix: it still treats the client-supplied string as the eventual on-disk filename and trusts it to

survive unchanged through every future code path. It does nothing about file *content* (no extension/type allow-list — an attacker can still upload a `.php`/`.exe` under a "safe" name) and nothing about the response still echoing that name back (the Information-disclosure finding from Task 2/3 survives unchanged). It also doesn't address "what if the filename, post-sanitization, collides with an existing file and overwrites it" — `secure_filename("../../app.py")` returns `"app.py"`, which would silently overwrite a same-named upload.

3. Correct, verified version: the Task 8 implementation — server-generated UUID filename, original name kept only as metadata, extension allow-list enforced. This is a class fix because no value derived from user input ever becomes a path component at all, regardless of what characters it contains, so there's no traversal string left to sanitize against, and it additionally closes the info-disclosure and unrestricted-extension gaps `secure_filename()` alone leaves open.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Answers

A. The `/upload` endpoint takes whatever filename the client sends and writes the file to disk using that name unchanged (`f.save(os.path.join(UPLOAD_DIR, f.filename))`). Because nothing strips `../` sequences, a client can send a filename that walks back out of the `uploads/` folder and lands anywhere else on the server the process can write to — turning a "save my file" feature into a way to overwrite arbitrary files on the host, up to and including the application's own source code.

B. Prompt used:

> "Here is a Flask upload endpoint: `f = request.files['file']; f.save(os.path.join(UPLOAD_DIR, f.filename))`. Rewrite it so that no value derived from the client-supplied filename can ever be used as a directory path component — reject path separators and `..`, allow-list file extensions, and store the file under a server-generated name. Show the corrected code only."

Result: matches the Task 8 implementation (`ext = os.path.splitext(f.filename)[1].lower()`; reject if not in `ALLOWED_EXTENSIONS`; `stored_name = f"{uuid.uuid4().hex}{ext}"`; `f.save(os.path.join(UPLOAD_DIR, stored_name))`). Verification: re-running the same traversal payload (`filename=../traversal-poc.txt`) that previously escaped `uploads/` now returns `{"original_name":"../traversal-poc.txt","saved":"65724a323e59454d99d97e3cea67fed1.txt"}` and the file lands safely inside `uploads/` under the random name — confirmed with `ls` showing nothing landed outside `uploads/` this time. The exploit fails.